EWB COURSES | AWS ARCHITECT ASSOCIATE

# AWS VPC Peering

## Private Connectivity Between Two VPCs

Student Project Report - Build, Test, Explain

| Field | Details |
|---|---|
| Name | Munmun Kapri |
| Batch | AWS Internship Batch 1 |
| Project | AWS VPC Peering: two VPCs, bastion host, route tables and security groups |
| AWS Region | Asia Pacific (Mumbai), ap-south-1 |
| Completion Date | 24 October 2026 |
| Course | EWB Courses, AWS Architect Associate (Project 01) |

## 1. Project Objective

The objective of this project was to create two Amazon Virtual Private Clouds (VPCs), connect them using VPC Peering, configure route tables and security-group rules, and verify private network communication using ICMP ping and SSH. The private application instances were accessed through a public bastion host rather than by assigning public IPv4 addresses to them.

## 2. Architecture Overview

VPC A (10.10.0.0/16) contains a public subnet with the bastion host and a private subnet with App-A. VPC B (10.20.0.0/16) contains a private subnet with App-B and has no internet gateway. The VPCs are connected by the peering connection ewb-a-b-peer, and the route tables in both VPCs send traffic for the other VPC CIDR to this peering connection. The two CIDR blocks do not overlap, which is a requirement for peering.

| Resource | Name | CIDR / Private IP | Resource ID |
|---|---|---|---|
| VPC A | ewb-vpc-a | 10.10.0.0/16 | vpc-0a2aff96800bf14d4 |
| VPC B | ewb-vpc-b | 10.20.0.0/16 | vpc-0e48c42314b0a926d |
| Public subnet | ewb-a-public | 10.10.1.0/24 | subnet-074d0e0fe81693cd4 |
| Private subnet A | ewb-a-private | 10.10.2.0/24 | subnet-0c4aed9a2448a7... [copy full ID from console] |
| Private subnet B | ewb-b-private | 10.20.1.0/24 | subnet-002ba8aa414d948e4 |
| Internet gateway | ewb-igw-a | Attached to VPC A | igw-0f2eb512b6af05679 |
| VPC peering | ewb-a-b-peer | Active | pcx-075bc5720eb176d88 |
| Bastion EC2 | ewb-bastion | 10.10.1.141; public 13.126.1.105 | i-0d34a977d5a0d9b42 |
| App EC2 A | ewb-app-a | 10.10.2.41; no public IPv4 | i-0e39e9baf8445be40 |
| App EC2 B | ewb-app-b | 10.20.1.101; no public IPv4 | i-01e819c9925d0e137 |

*AWS-assigned private IPs were used for the instances, so every instance IP and /32 security-group rule in this report uses these actual values instead of the workbook examples (10.10.1.10, 10.10.2.10, 10.20.1.10).*

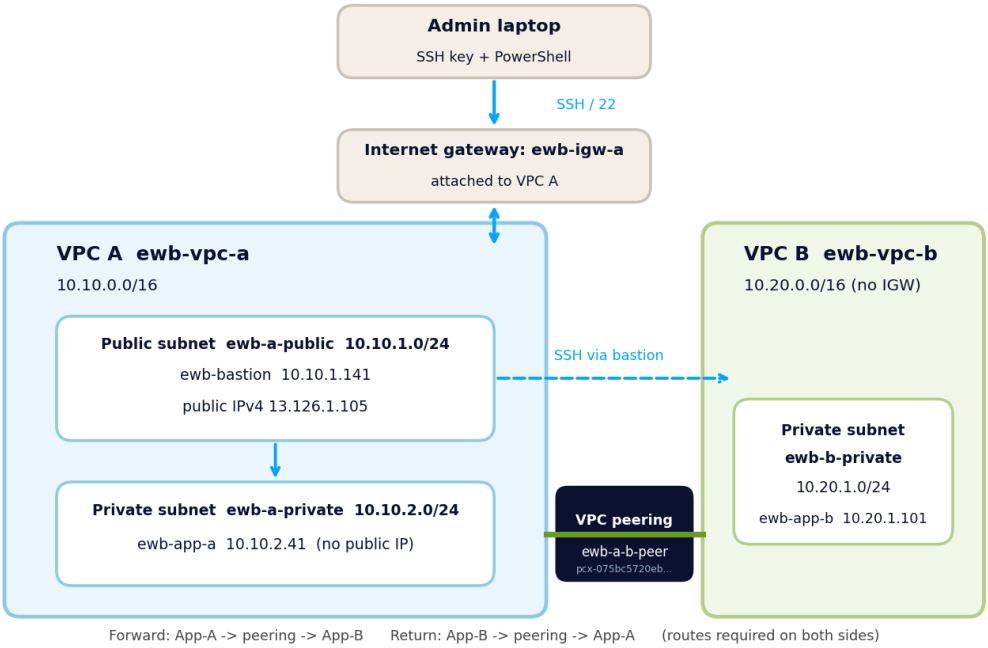



*Figure 1: VPC architecture and resource inventory*

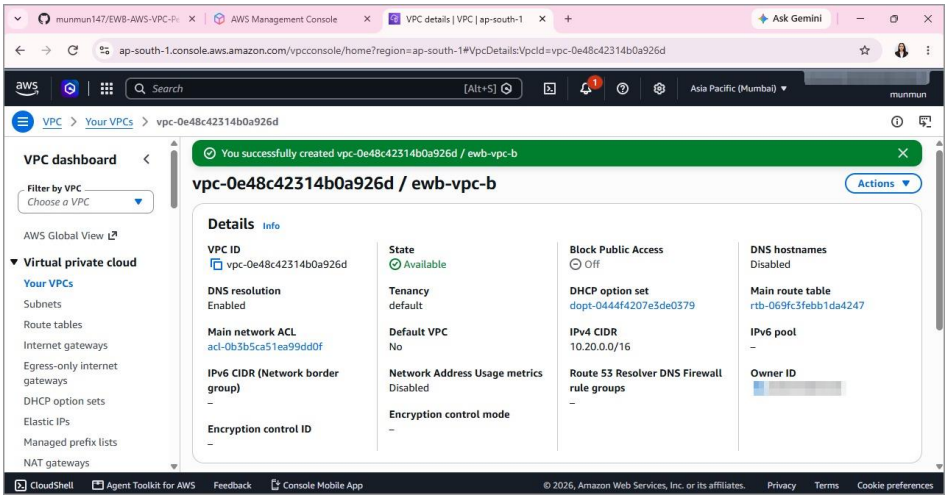



*Figure 2: ewb-vpc-b details (10.20.0.0/16, state Available, DNS resolution enabled)*

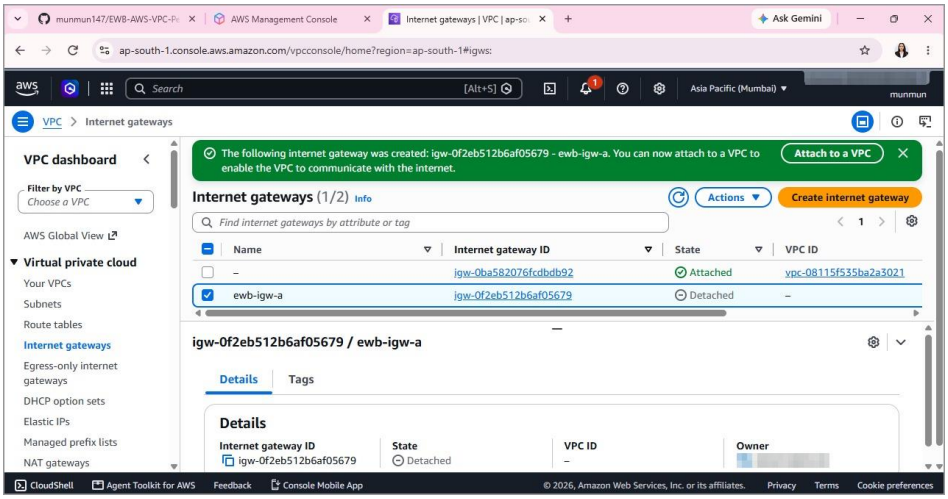



*Figure 3: Internet gateway ewb-igw-a created (attached to VPC A in the next step)*

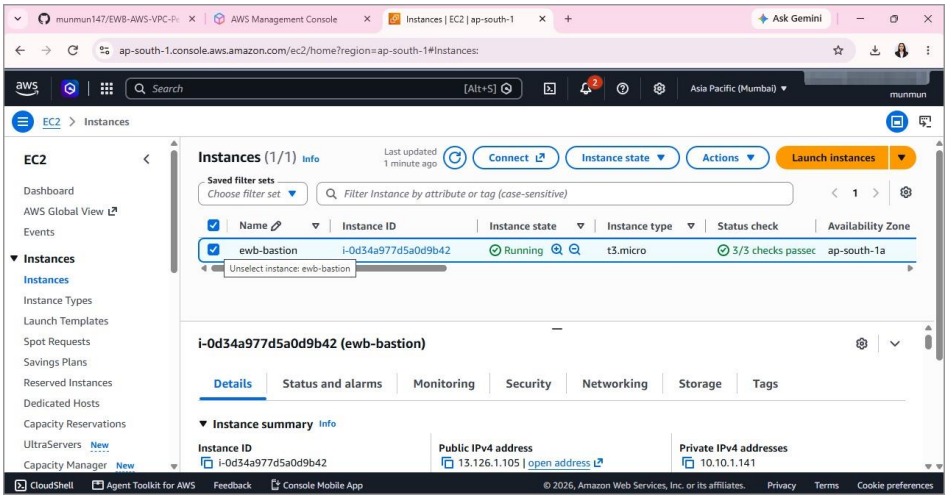



*Figure 4: ewb-bastion running, 3/3 status checks passed (private 10.10.1.141, public 13.126.1.105)*

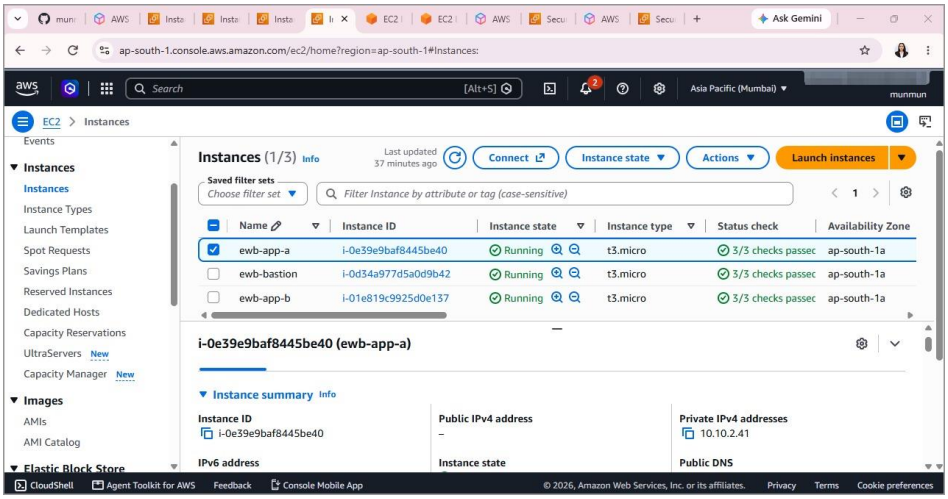



*Figure 5: ewb-app-a running (private 10.10.2.41, Public IPv4 address empty)*

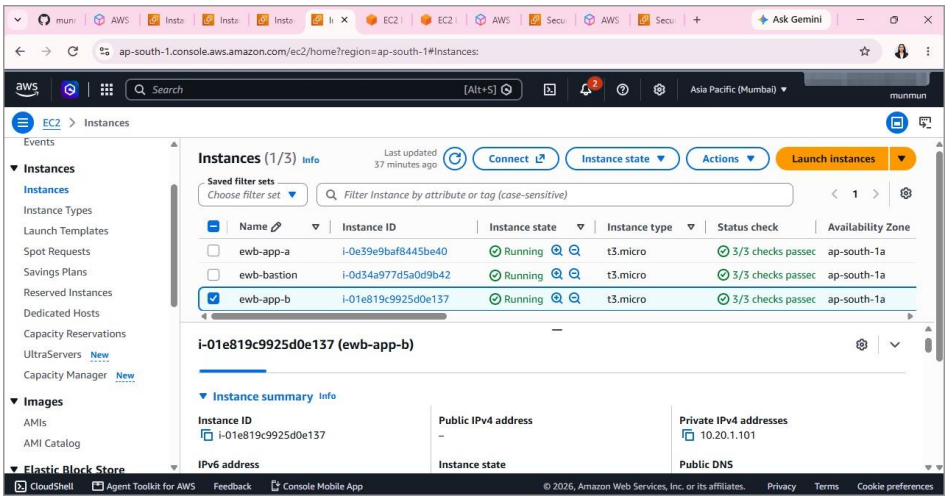



*Figure 6: ewb-app-b running (private 10.20.1.101, Public IPv4 address empty)*

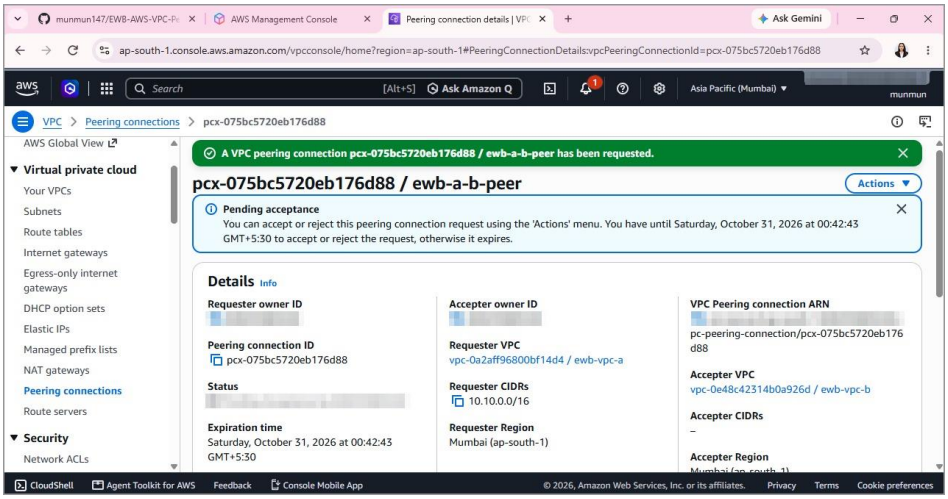



*Figure 7: Peering connection ewb-a-b-peer requested from ewb-vpc-a (10.10.0.0/16) to ewb-vpc-b*

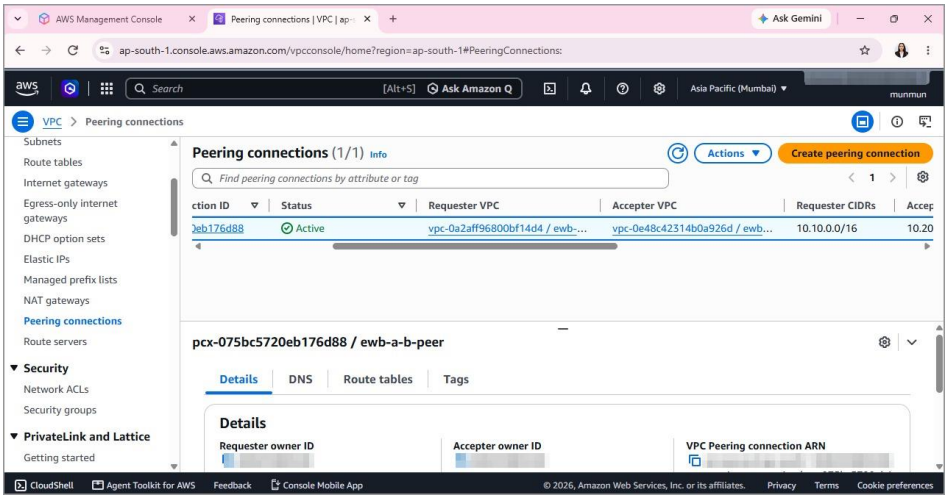



*Figure 8: Peering connection ewb-a-b-peer status Active (requester vpc-0a2aff96800bf14d4, accepter vpc-0e48c42314b0a926d, 10.10.0.0/16 and 10.20.0.0/16)*

*Figure 7 was captured at request time (Pending acceptance). After acceptance the connection became Active (Figure 8), and the Active peering routes are visible in Figure 10.*

## 3. Subnets and Route Tables

| Route table | Associated subnet | Route table ID | Routes |
|---|---|---|---|
| ewb-rt-a-public | ewb-a-public | rtb-0ab6142ae9d8415a1 | 10.10.0.0/16 > local; 0.0.0.0/0 > ewb-igw-a; 10.20.0.0/16 > ewb-a-b-peer |
| ewb-rt-a-private | ewb-a-private | rtb-0b65b15cb61496e9a | 10.10.0.0/16 > local; 10.20.0.0/16 > ewb-a-b-peer |
| ewb-rt-b-private | ewb-b-private | rtb-002f5546ab039c442 | 10.20.0.0/16 > local; 10.10.0.0/16 > ewb-a-b-peer |

**Forward path:** App-A (10.10.2.41) sends traffic for App-B (10.20.1.101) using the 10.20.0.0/16 route in ewb-rt-a-private, and the peering connection carries it to VPC B.

**Return path:** App-B replies using the 10.10.0.0/16 route in ewb-rt-b-private, which points back to the same peering connection. Without this return route, requests would arrive but replies would be lost.

**Public subnet route:** ewb-rt-a-public also has a 10.20.0.0/16 peering route so the bastion can open SSH connections to App-B. This more specific route is chosen over the 0.0.0.0/0 internet route.

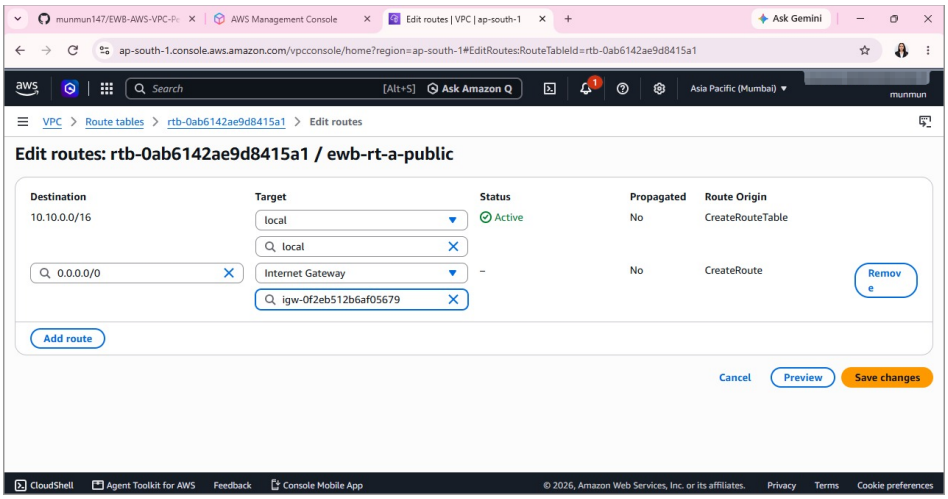



*Figure 9: Adding the default route 0.0.0.0/0 > ewb-igw-a to ewb-rt-a-public*

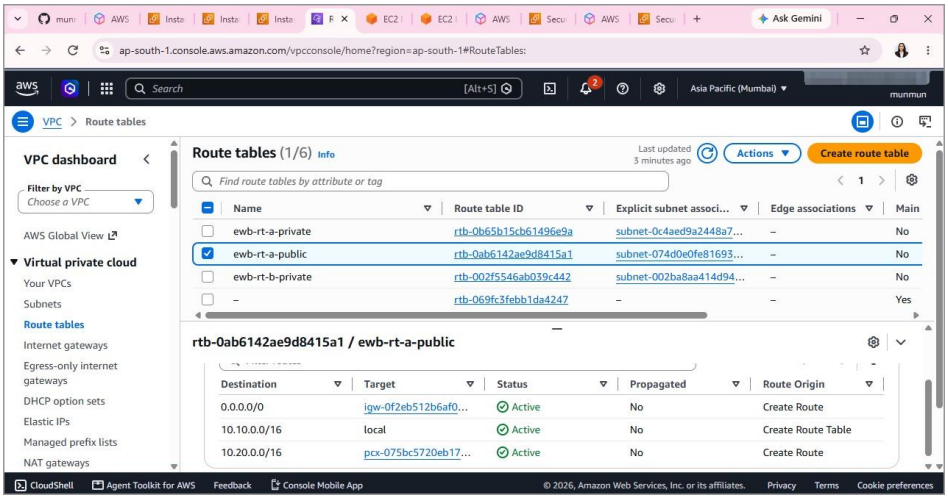



*Figure 10: Route table ewb-rt-a-public final routes: 0.0.0.0/0 > igw, 10.10.0.0/16 > local, 10.20.0.0/16 > pcx (all Active)*

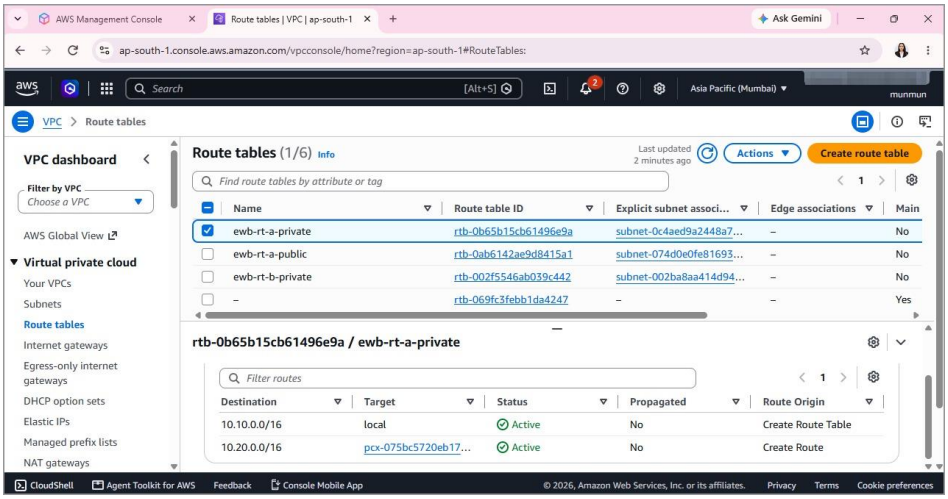



*Figure 11: Route table ewb-rt-a-private (associated with ewb-a-private): 10.10.0.0/16 > local and 10.20.0.0/16 > pcx (Active)*

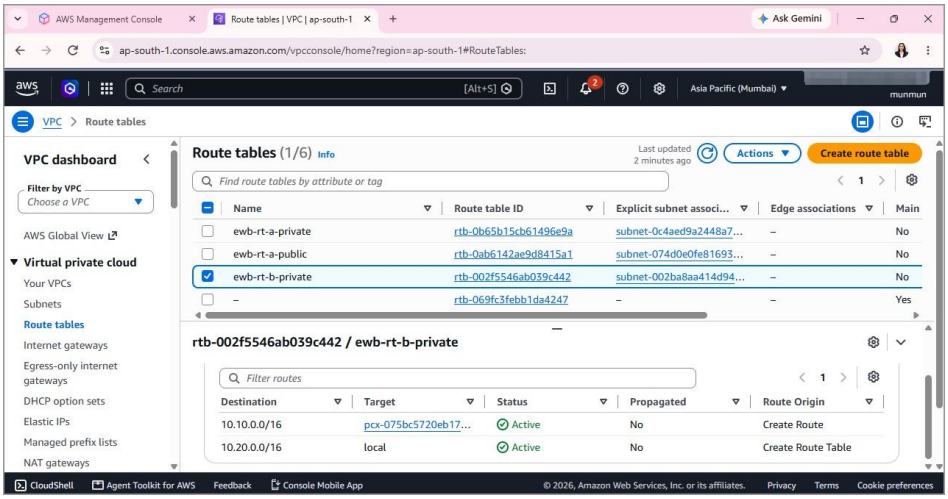



*Figure 12: Route table ewb-rt-b-private (associated with ewb-b-private): 10.10.0.0/16 > pcx (return route) and 10.20.0.0/16 > local (Active)*

## 4. Security Groups

Security groups control traffic reaching the EC2 instances. The bastion is the only SSH entry point from the internet. The private instances have no public IPv4 address, and SSH to App-A and App-B is allowed only from the bastion private IP (10.10.1.141/32). ICMP is allowed only between the two application hosts, and SSH or ICMP is never opened to 0.0.0.0/0.

| Security group | Inbound rule | Source | Purpose |
|---|---|---|---|
| ewb-bastion-sg | SSH / TCP 22 | Admin laptop public IPv4 /32 | Allow SSH from admin laptop only |
| ewb-sg-app-a | SSH / TCP 22 | 10.10.1.141/32 | Allow SSH from bastion |
| ewb-sg-app-a | All ICMP - IPv4 | 10.20.1.101/32 | Allow ping from App-B |
| ewb-sg-app-b | SSH / TCP 22 | 10.10.1.141/32 | Allow SSH from bastion |
| ewb-sg-app-b | All ICMP - IPv4 | 10.10.2.41/32 | Allow ping from App-A |

*Security groups are stateful, so replies to permitted traffic are allowed automatically. Default allow-all network ACLs were used.*

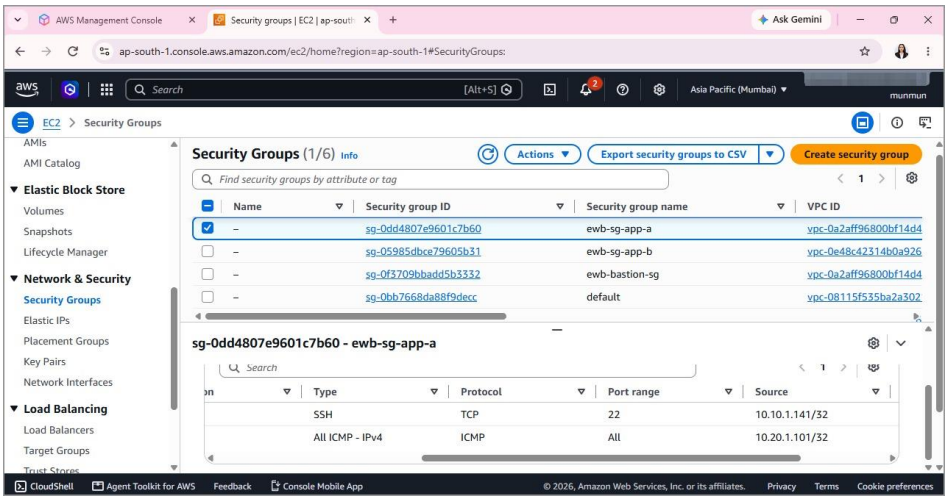



*Figure 13: App-A security group (ewb-sg-app-a) inbound rules: SSH 22 from 10.10.1.141/32 and All ICMP - IPv4 from 10.20.1.101/32*

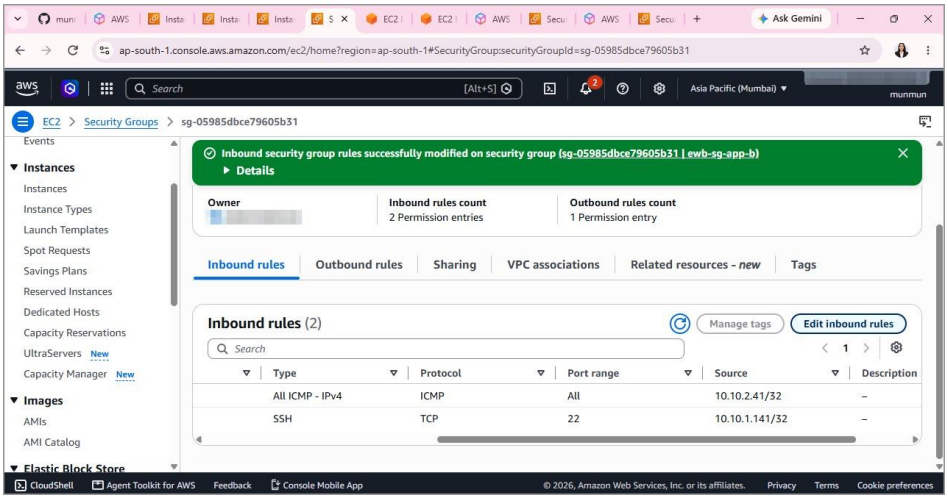



*Figure 14: App-B security group (ewb-sg-app-b) inbound rules: All ICMP - IPv4 from 10.10.2.41/32 and SSH 22 from 10.10.1.141/32*

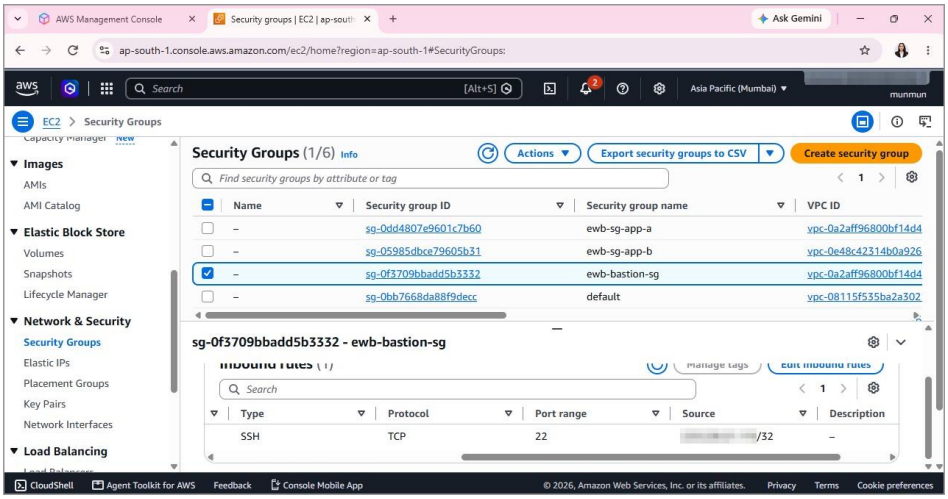



*Figure 15: Bastion security group (ewb-bastion-sg) inbound rule: SSH TCP 22 from the admin laptop public IPv4 /32 (IP blurred)*

# 5. Connectivity Testing

The following tests validate private connectivity. Every test uses private destination IP addresses.

## 5.1 SSH to Bastion

From Windows PowerShell, SSH was established to the bastion with the local private key: `ssh -i "$HOME\Downloads\ewb_key.pem" ec2-user@13.126.1.105`

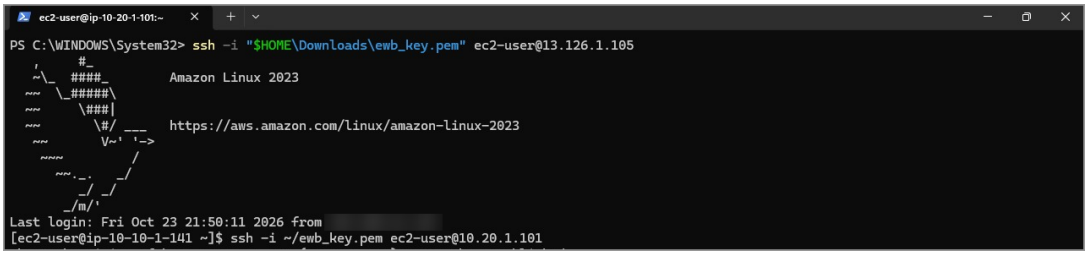



*Figure 16: Successful SSH login to ewb-bastion (prompt ip-10-10-1-141). The source IP in the last-login line is blurred.*

## 5.2 SSH from Bastion to App-A

The first SSH attempt from the bastion to App-A (10.10.2.41) was rejected with "UNPROTECTED PRIVATE KEY FILE" because the copied key had permissions 0664. After running chmod 400 on the key file, the same command succeeded and the prompt changed to ip-10-10-2-41.

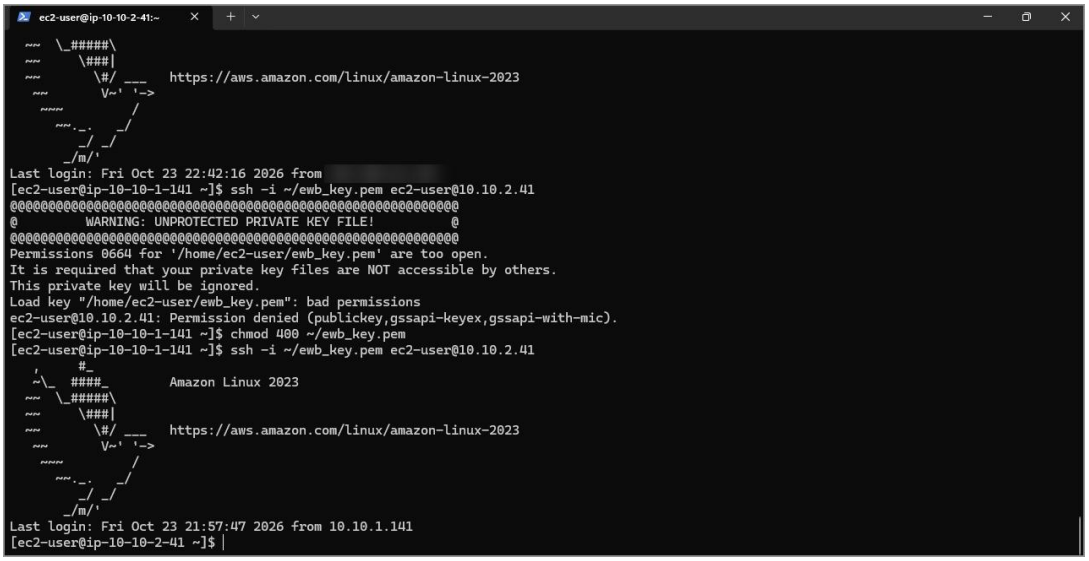



*Figure 17: Successful SSH login to App-A (10.10.2.41) after chmod 400 on the key file. The source IP in the last-login line is blurred.*

## 5.3 SSH from Bastion to App-B

From the bastion, SSH to App-B (10.20.1.101) was tested with `ssh -i ~/ewb_key.pem ec2-user@10.20.1.101`. The key had been copied from the laptop to the bastion with scp. The host key was accepted on first connection and the prompt changed to ip-10-20-1-101. This also confirms that the bastion route to VPC B and the return route from VPC B both work.

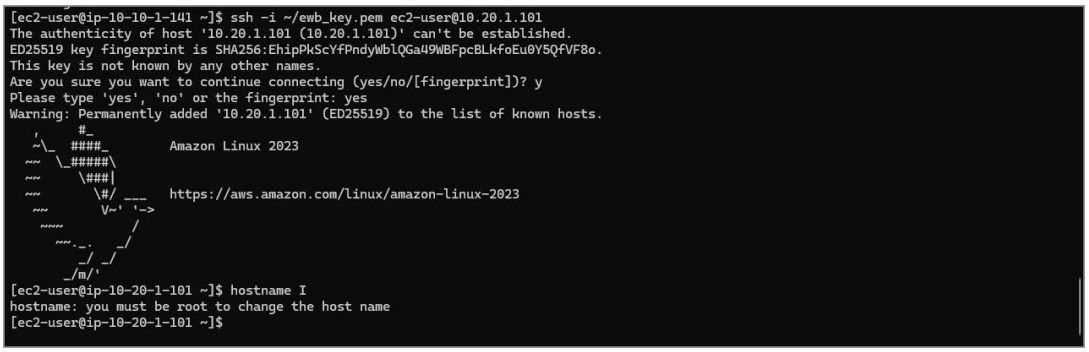



*Figure 18: Successful SSH login to App-B (10.20.1.101)*

## 5.4 Forward ping: App-A to App-B

Command: `ping -c 4 10.20.1.101`. Run from App-A (prompt ip-10-10-2-41). The first run lost all 4 packets because the ICMP inbound rule for 10.10.2.41/32 was not yet in place on ewb-sg-app-b. After the rule was added, the second run returned 4 of 4 replies with 0% packet loss (0.181 to 0.610 ms).

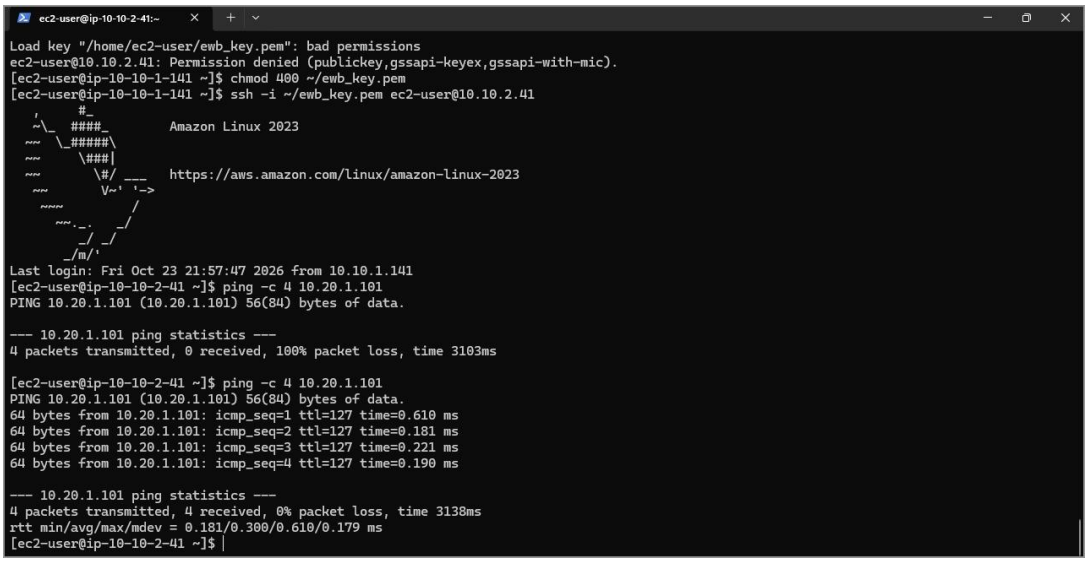



*Figure 19: App-A to App-B ping from App-A (ip-10-10-2-41): first run 100% packet loss, second run 4 of 4 replies*

## 5.5 Reverse ping: App-B to App-A

Command: `ping -c 4 10.10.2.41`. The command was run on App-B (hostname -I shows 10.20.1.101). The first attempt, made while the ICMP rule on ewb-sg-app-a was not in place, lost all packets. After the rule was applied the second attempt returned 4 of 4 replies with 0% packet loss and 0.150 to 0.174 ms round-trip time.

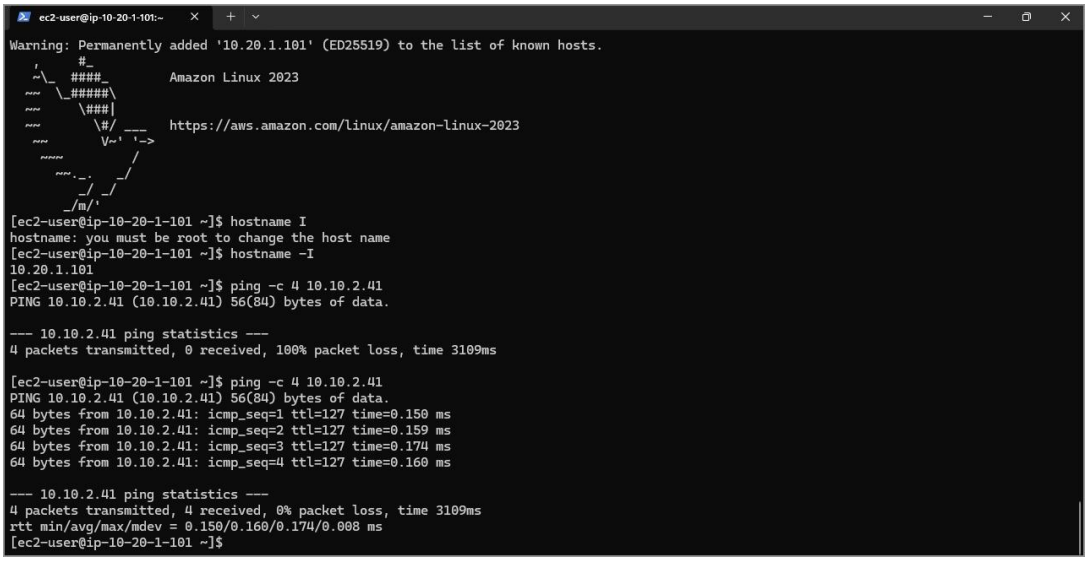



*Figure 20: App-B to App-A ping (hostname -I shows 10.20.1.101): failure, then success*

## 5.6 Controlled ICMP failure and recovery

Ping depends on both the peering routes and the ICMP inbound rule on the destination security group. With routes and peering already Active, the ping from App-A to App-B returned 100% packet loss while ewb-sg-app-b had no ICMP rule for 10.10.2.41/32 (Figure 21). After the rule was added, the same command returned 4 of 4 replies (Figure 22). The reverse test from App-B showed the same pattern (Figure 20). This shows that routing and the peering connection alone do not guarantee reachability; the security-group rule must also allow the traffic.




*Figure 21: Failure: ping App-A to App-B, 4 transmitted, 0 received, 100% packet loss*

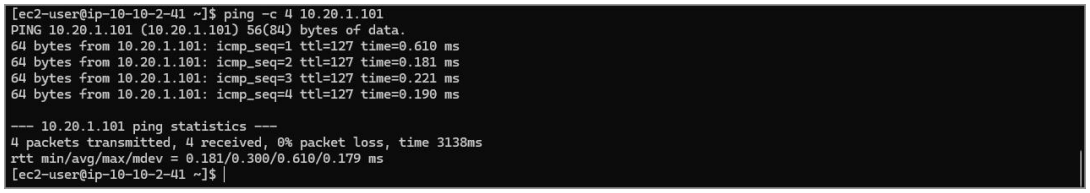



*Figure 22: Recovery: ping App-A to App-B, 4 transmitted, 4 received, 0% packet loss after the ICMP rule was added*

## 6. Baseline Test Before Peering

The baseline failure screenshot (ping between the two VPCs before the peering connection and routes were configured) was not captured during the initial setup, so no baseline evidence is included. The 100% packet-loss result in Figure 21 shows a security-group failure, not a pre-peering failure.

## 7. Cleanup and Security

After all evidence was saved, the lab resources were removed following the course cleanup checklist: terminate the three lab EC2 instances; check and delete remaining lab EBS volumes and snapshots; release any lab Elastic IP; delete the peering routes and the peering connection; delete the lab security groups after the network interfaces are gone; delete the lab subnets and custom route tables; detach and delete the internet gateway; and delete both lab VPCs. Then confirm that no project resources remain in ap-south-1 and review Billing for lingering charges.

**Private-key safety:** the private key was copied to the bastion temporarily for SSH testing from the bastion to the private hosts, and removed after testing with `rm -f ~/ewb_key.pem`. After testing, the key was deleted from the bastion. A later SSH attempt from the bastion failed with "Identity file ... No such file or directory" (Figure 23), confirming removal. The recommended alternative is SSH ProxyJump from the laptop, which avoids copying any key to the bastion. No private key, password or access token is included in this report.

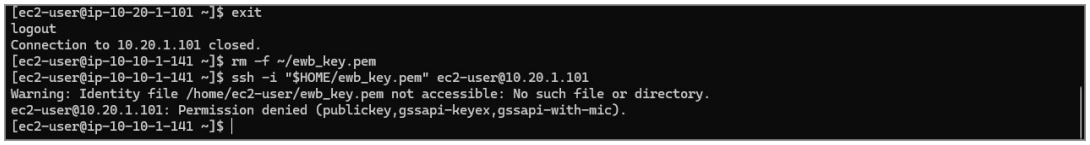



*Figure 23: Key removed from the bastion with rm -f ~/ewb_key.pem; the following SSH attempt fails because the file no longer exists*

**Cleanup evidence:** after all test evidence was saved, the lab resources were removed. The screenshots below show the three EC2 instances terminated, the lab security groups deleted, the lab subnets deleted, the internet gateway detached from the VPC, and the EC2 dashboard in ap-south-1 showing 0 running instances, 0 volumes, 0 snapshots and 0 Elastic IPs.

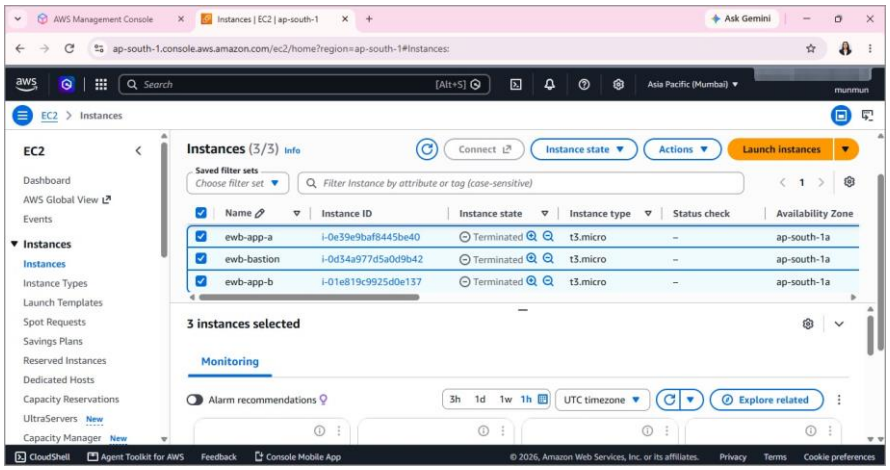



*Figure 24: Cleanup: ewb-app-a, ewb-bastion and ewb-app-b in Terminated state*

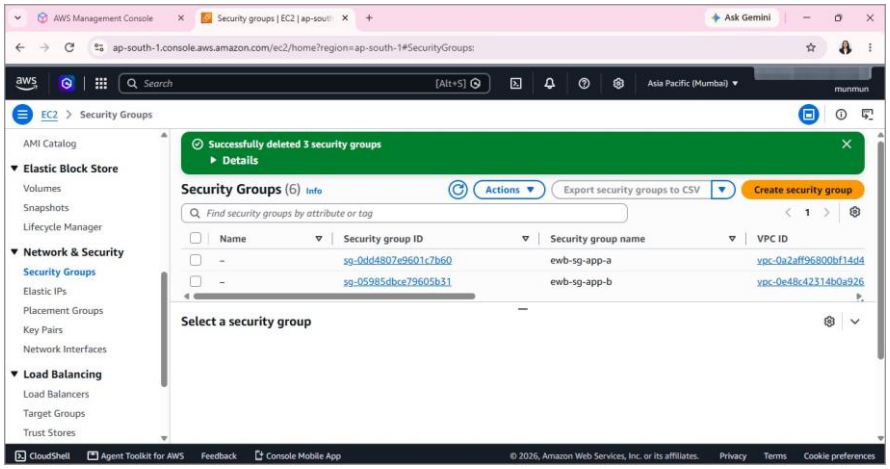



*Figure 25: Cleanup: lab security groups deleted ("Successfully deleted 3 security groups"; table not yet refreshed)*

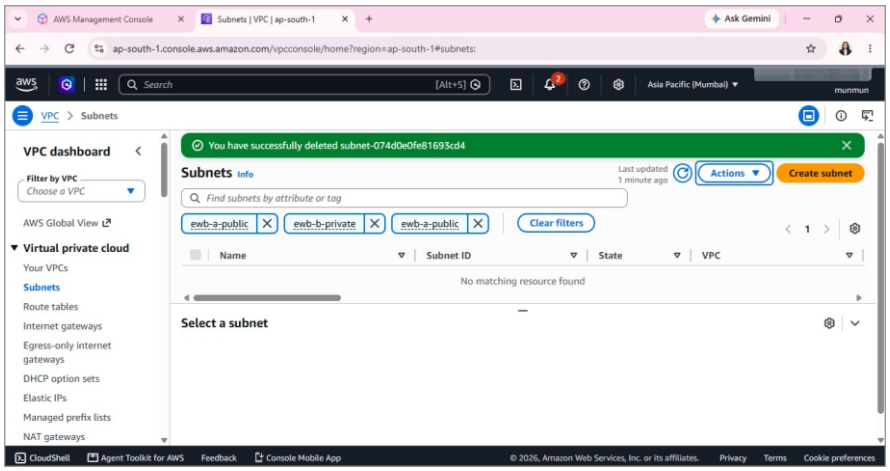



*Figure 26: Cleanup: lab subnet deleted (ewb-a-public, ewb-b-private no longer found)*

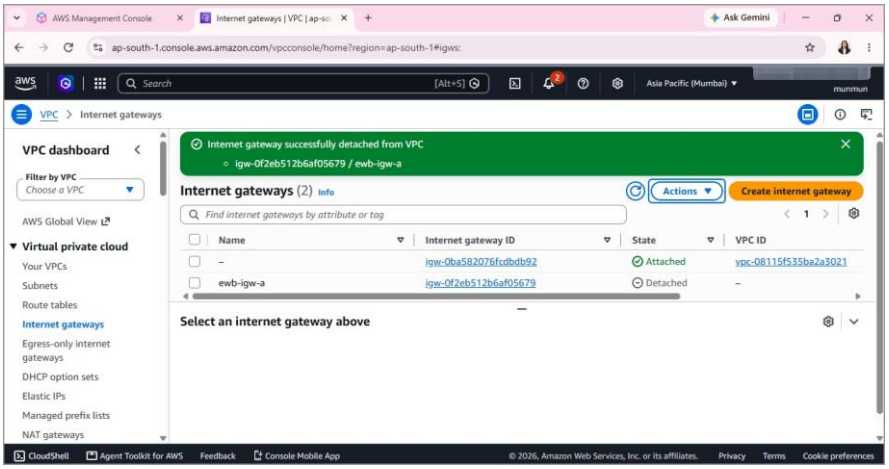



*Figure 27: Cleanup: internet gateway ewb-igw-a successfully detached from the VPC*

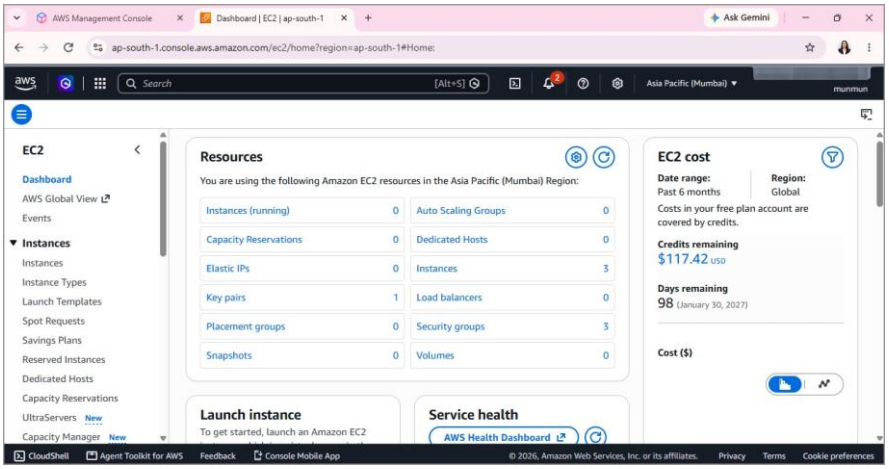



*Figure 28: Cleanup verification: EC2 dashboard, 0 running instances, 0 Elastic IPs, 0 snapshots, 0 volumes*

## 8. What I Learned

- A VPC peering connection enables private communication between two VPCs only when it is Active and routes are configured on both sides.
- Both the forward route and the return route are required for two-way connectivity.
- Security-group inbound rules allow or block specific traffic such as SSH and ICMP, as shown by the controlled failure and recovery test.
- Private EC2 instances can be administered through a bastion host without assigning them public IPv4 addresses.
- Private keys need restrictive file permissions (chmod 400) and should not be left on a bastion host; ProxyJump avoids copying the key at all.
- Evidence must be saved before deleting lab resources, and cleanup must be verified to avoid unnecessary charges.

## Appendix: Viva Preparation

**1. Why is the return route needed?** A peering route is one-directional. App-B must know that 10.10.0.0/16 is reached through the peering connection, otherwise it cannot send replies.

**2. Why is the App-B SSH source the bastion private IP?** The bastion opens the connection to App-B over the peering link, so App-B sees the bastion private address (10.10.1.141) as the source.

**3. Why does Active peering not guarantee a successful ping?** Peering is only the link. Route tables must point to it on both sides, and the destination security group must allow ICMP.

**4. Can VPC B use VPC A internet gateway over peering?** No. Peering does not provide transitive access to an internet gateway, so VPC B has no internet access through VPC A.